In our paper we also use a weaker definition of verified error (which can be bounded using a surrogate loss, which will be presented in Section 3.3), where we consider each attack target individually:

$$\text{Targeted Verified Error} := 1 - \frac{|\{(k,j)|M(\boldsymbol{x}_k, j) > 0\}|}{N \times (C-1)}. \tag{7}$$

3.2 Linear outer bounds for an ensemble

We denote $M_t(\boldsymbol{x}_k, j)$ as the margin for a base model $f_t(\boldsymbol{x})$. We first note that the lower bound on margin $M_t(\boldsymbol{x}_k, j)$ is unnormalized and not scale-invariant. Enlarging the output of $f_t(\boldsymbol{x})$ by a constant factor does not affect the robustness of $f_t(\boldsymbol{x})$, but the margin will also be scaled correspondingly. Directly maximizing this unnormalized margin does not lead to better robustness. Intuitively, the most important factor of the robustness of model $f_t(\boldsymbol{x})$ is the available budget (reflected as the term $\widehat{\mathbf{c}}_{t,j}$) divided by the sensitivity with respect to the input (reflected as $\|\widehat{\mathbf{L}}_{t,(j,:)}\|_q$), rather than the absolute value of the margin. When we use this lower bound on margins as a surrogate to compare the robustness across different models, their margins should be within a similar range to make this comparison meaningful. To take this into account, we can normalize $M_t(\boldsymbol{x}_k, j)$ by dividing it with a normalizing factor $Z_t := \frac{1}{N(C-1)} \sum_{k=i}^{N} \sum_{j \in [C-1]} \widehat{\mathbf{c}}_{t,j}^k$, the average $\widehat{\mathbf{c}}$ over all examples and classes. The normalized lower bound on margin is defined as:

$$\widetilde{M}_t(\boldsymbol{x}_k, j) := -\epsilon \|\widetilde{\mathbf{L}}_{t,(j,:)}\|_q + \widetilde{\mathbf{c}}_{t,j}, \tag{8}$$
$$\text{where } \widetilde{\mathbf{L}}_{t,(j,:)} := \frac{1}{Z_t} \widehat{\mathbf{L}}_{t,(j,:)}, \quad \widetilde{\mathbf{c}}_{t,j} := \frac{1}{Z_t} \widehat{\mathbf{c}}_{t,j}.$$

This is equivalent to applying a constant factor $\frac{1}{Z_t}$. The normalized model, denoted as $\widetilde{f}_t(\boldsymbol{x}) := \frac{1}{Z_t} f_t(\boldsymbol{x})$, will be used for our ensemble. We can use other normalizing schemes as long as they roughly keep each model's margin in a similar magnitude to ease the optimization.

We define the model ensemble of a set of T neural networks as $F(\boldsymbol{x}) = \sum_{t=1}^{T} \alpha_t \widetilde{f}_t(\boldsymbol{x})$, where $0 \le \alpha_t \le 1$, $\sum_{t=1}^{T} \alpha_t = 1$. α_t is the coefficient for classifier $\widetilde{f}_t(x)$. In our setting, each neural network classifier has been trained with certificates (Def. 3.2), and we want to further enhance them using carefully selected ensemble weights. Because they are linearly combined, we can give a linear upper and lower bound for $F(\boldsymbol{x})$, by linearly combining the upper and lower bounds given by Eq. (1). Again, for a training example $(\boldsymbol{x}_k, y_k)$, the lower bound on the margin $[\widehat{F}(\boldsymbol{x})]_j$ of $F(\boldsymbol{x})$ for the class $j \neq y_k$ inside $B_p(\boldsymbol{x}_k, \epsilon)$ is:

$$\left(\sum_{t=1}^{T} \alpha_t \widetilde{\mathbf{L}}_{t,(j,:)} \right) \Delta \boldsymbol{x} + \sum_{t=1}^{T} \alpha_t \widetilde{\mathbf{c}}_{t,j} \le [\widehat{F}(\boldsymbol{x})]_j \tag{9}$$

Analogous to Eq. (4), these bounds are *guaranteed* for any $\boldsymbol{x} = \boldsymbol{x}_k + \Delta \boldsymbol{x}$, $\Delta \boldsymbol{x} \in B_p(0, \epsilon)$. $\widetilde{\mathbf{L}}_t \in \mathbb{R}^{(C-1) \times d}, \widetilde{\mathbf{c}}_t \in \mathbb{R}^{C-1}$ are computed for model $\widetilde{f}_t(\boldsymbol{x})$ using Proposition 3.1 with a similar network transformation as in Eq. (2) and (3) for each $\widetilde{f}_t(\boldsymbol{x})$. Note that each $f_t(\boldsymbol{x})$ can have *completely different internal structure* (number of layers, number of neurons, architecture, etc) and be trained using different schemes, but their corresponding $\widetilde{\mathbf{L}}_t$ and $\widetilde{\mathbf{c}}_t$ have the same dimension. For the ensemble, the normalized lower bounds of margin is the following:

$$M(\boldsymbol{x}_k, j) := -\epsilon \| \sum_{t=1}^{T} \alpha_t \widetilde{\mathbf{L}}_{t,(j,:)} \|_q + \sum_{t=1}^{T} \alpha_t \widetilde{\mathbf{c}}_{t,j} \tag{10}$$

Main Idea: Why an ensemble can improve robustness? To enhance robustness via an ensemble, we want to find an optimal $\boldsymbol{\alpha}$, such that $M(\boldsymbol{x}_k, j)$ is maximized for all training examples $k \in [N]$ and target classes $s(j, y_k), j \in [C-1]$. A model $\widetilde{f}_t(\boldsymbol{x})$ lacking of robustness often has large $\|\widetilde{\mathbf{L}}_{t,(j,:)}\|_q$; by combining $\widetilde{\mathbf{L}}_{t,(j,:)}$ of different models with optimal weights, we hope that some noises in $\widetilde{\mathbf{L}}_{t,(j,:)}$ can be canceled out and $\sum_{t=1}^{T} \alpha_t \widetilde{\mathbf{L}}_{t,(j,:)}$ has a smaller q-norm than $\widetilde{\mathbf{L}}_{t,(j,:)}$. When there are very limited number of models for selection, a naive ensemble (with all $\alpha_t = \frac{1}{T}$) cannot guarantee to achieve this goal. Instead, **RobBoost** optimally select α_t based on maximizing a surrogate loss on normalized lower bounds of margins.

In Figure 1, we plot matrix $\widetilde{\mathbf{L}}$ for a naturally trained and a robust model (adversarially trained using (Madry et al., 2018)) to show the intuition behind RobBoost. Strikingly, $\widetilde{\mathbf{L}}$ has a quite interpretable pattern, especially on the adversarially trained model – $\widetilde{\mathbf{L}}$ is surprisingly sparse and the model output is only sensitive to changes on the pixels of the digit "1"; on the other hand, the naturally trained model has a lot of random noise around the "1" in the center, and is sensitive to many irrelevant background pixel changes. Our aim is thus to make the matrix $\widetilde{\mathbf{L}}$ less "noisy" through an careful ensemble: $\widetilde{\mathbf{L}} = \sum_{t=1}^{T} \alpha_t \widetilde{\mathbf{L}}_t$.

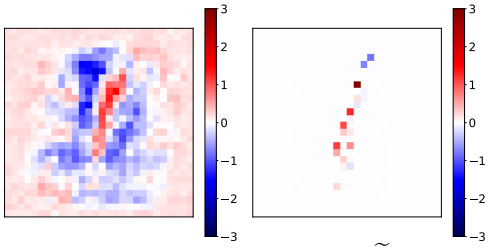

Figure 1: Left: visualization of matrix $\widetilde{\mathbf{L}}$ for a naturally trained MNIST model. Right: visualization of matrix $\widetilde{\mathbf{L}}$ for an adversarially trained MNIST model. Both models are 4-layer MLPs with 1024 neurons per layer. We use an image of digit "1" ($i = 1$) as the input, and set attack target to "2" ($s(j, i) = 2$), $\epsilon = 0.05$.

3.3 The RobBoost Loss Function

We first define the following RobBoost loss to maximize model's lower bound of margin via ensemble, across all examples and classes:

$$\begin{aligned} \text{minimize } \ g(\boldsymbol{\alpha}) = \sum_{k\in[N]} \sum_{j\in[C-1]} \max(1 - M(\boldsymbol{x}_k, j), 0) \\ \text{s.t. } \ 0 \le \boldsymbol{\alpha} \le 1, \quad \sum\nolimits_{t=1}^{T} \alpha_i = 1 \end{aligned} \tag{11}$$

where $\boldsymbol{\alpha} \in \mathbb{R}^T$ is the vector of weights for each model $\widetilde{f}_t(\boldsymbol{x})$, and $M(\boldsymbol{x}_k, j)$ is defined as in Eq. (10). This is a hinge-style surrogate loss of the 0-1 error defined in (7) that encourages large margin. We aim to decrease (7) by using an optimally weighted ensemble.

In this paper, we focus on the most common threat model where ℓ_∞ adversarial distortion ($p = \infty, q = 1$) is applied (ℓ_∞-RobBoost). In this case, Eq. (10) is a summation of absolute values:

$$M(\boldsymbol{x}_k, j) \coloneqq -\epsilon \sum_{i=1}^{d} |\sum_{t=1}^{T} \alpha_t \widetilde{\mathbf{L}}^k_{t,(j,i)}| + \sum_{t=1}^{T} \alpha_t \widetilde{\mathbf{c}}^k_{t,j} \tag{12}$$

Here we explicitly write out the dependency of k in $\widetilde{\mathbf{L}}$ and $\widetilde{\mathbf{c}}$. When j, k is fixed, we can define $N \times (C-1)$ new matrices $\overline{\mathbf{A}} \in \mathbb{R}^{d\times T}$ with reordered indices and absorb ϵ into it: $\overline{\mathbf{A}}^{j,k}_{(i,t)} \coloneqq \epsilon \widetilde{\mathbf{L}}^k_{t,(j,i)}$. And similarly $\bar{\mathbf{c}}^{j,k}_t \coloneqq \widetilde{\mathbf{c}}^k_{(t,j)}$ such that Eq. (12) can be rewritten as:

$$M(\boldsymbol{x}_k, j) = -\|\overline{\mathbf{A}}^{j,k}\boldsymbol{\alpha}\|_1 + \bar{\mathbf{c}}^{j,k\top}\boldsymbol{\alpha}$$

Note that for each example k and each target class j we have an $\overline{\mathbf{A}}$ and a $\bar{\mathbf{c}}$. Then Eq. (11) becomes:

$$\begin{aligned} \text{minimize } \ g(\boldsymbol{\alpha}) = \\ \sum_{k\in[N]} \sum_{j\in[C-1]} \max(\|\overline{\mathbf{A}}^{j,k}\boldsymbol{\alpha}\|_1 - \bar{\mathbf{c}}^{j,k\top}\boldsymbol{\alpha} + 1, 0) \\ \text{s.t. } \ 0 \le \boldsymbol{\alpha} \le 1, \quad \sum\nolimits_{t=1}^{T} \alpha_i = 1 \end{aligned} \tag{13}$$

Since the objective is non-smooth, piece-wise linear and low-dimensional, we propose to use coordinate descent to efficiently solve this minimization problem.

Solving ℓ_∞-RobBoost using coordinate-descent. In coordinate descent, we aim to solve the following one variable optimization problem with a randomly selected coordinate t:

$$\begin{aligned} \alpha_t^* &= \arg\min_{0\le\alpha_t\le 1} g(\alpha_t), \text{ where} \\ g(\alpha_t) &= \sum_{k\in[N]} \sum_{j\in[C-1]} \max(\|\overline{\mathbf{A}}^{j,k}\boldsymbol{\alpha}\|_1 - \bar{\mathbf{c}}^{j,k\top}\boldsymbol{\alpha} + 1, 0) \\ &= \sum_{k\in[N]} \sum_{j\in[C-1]} \max(\|\alpha_t \overline{\mathbf{A}}^{j,k}_{(:,t)} + \overline{\mathbf{A}}^{j,k}(\boldsymbol{\alpha} - \alpha_t \boldsymbol{e}_t)\|_1 - \\ &\quad \bar{\mathbf{c}}^{j,k\top}(\boldsymbol{\alpha} - \alpha_t \boldsymbol{e}_t) - \bar{\mathbf{c}}^{j,k}\alpha_t + 1, 0) \\ &\coloneqq \sum_{k\in[N]} \sum_{j\in[C-1]} \max(\|\alpha_t \overline{\mathbf{A}}^{j,k}_{(:,t)} + \boldsymbol{v}^{j,k}\|_1 + \\ &\quad u^{j,k} - \bar{\mathbf{c}}^{j,k\top}\alpha_t + 1, 0) \\ &\text{s.t.} \sum\nolimits_{t=1}^{T} \alpha_i = 1 \end{aligned} \tag{14}$$

where $\boldsymbol{v}^{j,k} = \overline{\mathbf{A}}^{j,k}(\boldsymbol{\alpha} - \alpha_t \boldsymbol{e}_t)$ and $u^{j,k} = \bar{\mathbf{c}}^{j,k\top}(\boldsymbol{\alpha} - \alpha_t \boldsymbol{e}_t)$ are constants with respect to α_t. While the box constraint $0 \le \alpha_t \le 1$ can be easily handled by coordinate-descent, the challenge is the constraint $\sum_{t=1}^{T} \alpha_i = 1$, which needs to be directly enforced during the coordinate descent procedure. Suppose we have maintained $\sum_{t=1}^{T} \alpha_i = 1$ before update, when updating variable α_t, we enforce this constraint by scaling all other $\alpha_s, s \ne t$ by a factor of $\frac{1-\alpha_t}{\sum_{s\ne t}\alpha_s}$ such that $\alpha_t + \sum_{s\ne t}\alpha_s \equiv 1$. In other words, we redefine $g(\alpha_t)$ as:

$$\begin{aligned} g(\alpha_t) &= \sum_{k\in[N]} \sum_{j\in[C-1]} \max(\|\alpha_t \overline{\mathbf{A}}^{j,k}_{(:,t)} + \frac{1-\alpha_t}{\sum_{s\ne t}\alpha_s} \boldsymbol{v}^{j,k}\|_1 \\ &+ \frac{1-\alpha_t}{\sum_{s\ne t}\alpha_s} u^{j,k} - \bar{\mathbf{c}}^{j,k\top}\alpha_t + 1, 0) \\ &\coloneqq \sum_{k\in[N]} \sum_{j\in[C-1]} \max(\|\alpha_t \boldsymbol{\omega}^{j,k} + \boldsymbol{\nu}^{j,k}\|_1 + \gamma^{j,k}\alpha_t + \mu^{j,k} + 1, 0) \\ &\coloneqq \sum_{k\in[N]} \sum_{j\in[C-1]} \max(g^{j,k}(\alpha_t), 0) \end{aligned} \tag{15}$$

where $\boldsymbol{\omega}^{j,k} \coloneqq \overline{\mathbf{A}}^{j,k}_{(:,t)} - \frac{1}{\sum_{s\ne t}\alpha_s}\boldsymbol{v}^{j,k}$, $\boldsymbol{\nu}^{j,k} = \frac{1}{\sum_{s\ne t}\alpha_s}\boldsymbol{v}^{j,k}$ and $\mu^{j,k} = \frac{u^{j,k}}{\sum_{s\ne t}\alpha_s} + 1$, $\gamma^{j,k} = -\bar{\mathbf{c}}^{j,k\top} - \frac{1}{\sum_{s\ne t}\alpha_s}$. Now the summation constraint has been removed, and for any $0 \le \alpha_t \le 1$ we guarantee $\sum_{t=1}^{T} \alpha_i = 1$. Each term $g^{j,k}(\alpha_t)$ is a bounded one-dimensional piece-wise linear function within domain $0 \le \alpha_t \le 1$. The term $\|\alpha_t \boldsymbol{\omega}^{j,k} + \boldsymbol{\nu}^{j,k}\|_1$ contains d linear terms inside absolute value so there are at most d pieces. The minima of this term must be on the end of one piece, or at the boundary 0 or 1. Solving for α_t in d equations $\alpha_t \boldsymbol{\omega}^{j,k}_l + \boldsymbol{\nu}^{j,k}_l = 0, l \in [d]$ gives us the locations of the end points of these pieces in $O(d)$ time, denoted as $r^{j,k}_1, \cdots, r^{j,k}_d$. We consider the worst case where all d points lie in $(0, 1)$, and for convenience we denote $r^{j,k}_0 = 0$, $r^{j,k}_{d+1} = 1$. Now the challenge remains to efficiently evaluate

$g^{j,k}(\alpha_t)$ at these d endpoint and two boundaries, each in $O(1)$ time. We first sort $r_1^{j,k}, \cdots, r_d^{j,k}$ in ascending order as $r_{\pi(1)}^{j,k}, \cdots, r_{\pi(d)}^{j,k}$, where π is the permutation of sorting and we additionally define $\pi(0) := 0$. Then, we start with $r_0^{j,k} = 0$, and check the sign of $z_l^{j,k}(\alpha_t) := \alpha_t \boldsymbol{\omega}_l^{j,k} + \boldsymbol{\nu}_l^{j,k}$ at $\alpha_t = r_0^{j,k}$ for each $l \in [d]$. We define two sets indicating the sign of $z_l^{j,k}$:

$$J_+^{j,k} = \{l | z_l^{j,k}(0) > 0, \text{ or } z_l^{j,k}(0) = 0, \boldsymbol{\omega}_l^{j,k} > 0\} \quad (16)$$

$$J_-^{j,k} = \{l | z_l^{j,k}(0) < 0, \text{ or } z_l^{j,k}(0) = 0, \boldsymbol{\omega}_l^{j,k} < 0\} \quad (17)$$

They can be formed in $O(d)$ time. Then we define the effective slope and intercept at the point r_0 as:

$$a_0^{j,k} = \sum_{l \in J_+^{j,k}} \boldsymbol{\omega}_l^{j,k} - \sum_{l \in J_-^{j,k}} \boldsymbol{\omega}_l^{j,k} + \gamma^{j,k} \quad (18)$$

$$b_0^{j,k} = \sum_{l \in J_+^{j,k}} \boldsymbol{\nu}_l^{j,k} - \sum_{l \in J_-^{j,k}} \boldsymbol{\nu}_l^{j,k} + \mu^{j,k} \quad (19)$$

and then we can evaluate $g^{j,k}(r_0^{j,k}) = a_0^{j,k} \times r_0^{j,k} + b_0^{j,k}$. For $i = \{1, \cdots, N'\}$, we evaluate $g^{j,k}(r_{\pi(i)}^{j,k})$ one by one. Assuming we already obtained $a_m^{j,k}$ and $b_m^{j,k}$ $(m < d)$ and evaluated $g^{j,k}(r_{\pi(m)}^{j,k}) = a_m^{j,k} r_{\pi(m)}^{j,k} + b_m^{j,k}$. We can then recursively define $a_{m+1}^{j,k}$ and $b_{m+1}^{j,k}$ as:

$$\begin{aligned} a_{m+1}^{j,k} = a_m^{j,k} &+ I(\pi(m+1) \in J_-^{j,k}) \cdot 2\boldsymbol{\omega}_{\pi(m+1)}^{j,k} \\ &- I(\pi(m+1) \in J_+^{j,k}) \cdot 2\boldsymbol{\omega}_{\pi(m+1)}^{j,k} \end{aligned} \quad (20)$$

$$\begin{aligned} b_{m+1}^{j,k} = b_m^{j,k} &+ I(\pi(m+1) \in J_-^{j,k}) \cdot 2\boldsymbol{\nu}_{\pi(m+1)}^{j,k} \\ &- I(\pi(m+1) \in J_+^{j,k}) \cdot 2\boldsymbol{\nu}_{\pi(m+1)}^{j,k} \end{aligned} \quad (21)$$

$I(\cdot)$ is an indicator function. We keep maintaining the slope and intercept for the next linear piece, when the sign of term $\pi(m+1)$ just changed. This update only takes $O(1)$ time. For $m = d+1$, we reached the boundary 1 and evaluate $g^{j,k}(1) = a_d^{j,k} \times 1 + b_d^{j,k}$.

For the final sum of surrogate losses, we merge sort all $r_i^{j,k}$ for all (j,k) terms into a new vector r with $N \times (C-1) \times d$ elements, also maintain a mapping $p(i) \to (j,k)$ which maps an element r_i into the summation term (j,k) it comes from. Our final algorithm evaluates the objective function on all r_i and using the maintained effective slope $a^{j,k}$ and intercept $b^{j,k}$ for each term in summation, and update $a^{p(i)}$ and $b^{p(i)}$ using (20) and (21). Additionally, we need to consider up to NC additional linear pieces introduced by the max function. We list the full algorithm in appendix in Algorithm 1. In each iteration of coordinate descent, we randomly choose a coordinate t, and obtain the best value α_t^* (14) to minimize the loss and set $\alpha_t \leftarrow \alpha_t^*$. Then we choose another coordinate and repeat. We observe that 2 to 3 epochs (each epoch visits all coordinate once) are sufficient to find a good solution.

3.4 RobBoost in Gradient Boosting

Gradient Boosting builds a strong model $F(\boldsymbol{x})$ by iteratively training and combining weak models:

$$F_{m+1}(\boldsymbol{x}) = F_m(\boldsymbol{x}) + f_{m+1}(\boldsymbol{x})$$

where $F_m(\boldsymbol{x}) = \sum_{t=1}^m f_t(\boldsymbol{x})$ is kept unchanged and we train $f_{m+1}(\boldsymbol{x})$ to reduce a certain loss function on $F_{m+1}(\boldsymbol{x})$. Unlike the setting we discussed in Section 3.3 where all base models are given and fixed, here we are allowed to update the model parameters of the last base model, with previous models frozen. Similar to Eq. (10), we can write the margin for example $\boldsymbol{x}_k$ target class j for the setting of gradient boosting:

$$\begin{aligned} M(\boldsymbol{x}_k, j) := &- \epsilon \| \sum\nolimits_{t=1}^m \widehat{\mathbf{L}}_t + \widehat{\mathbf{L}}_{m+1} \|_q \\ &+ \sum\nolimits_{t=1}^m \widehat{\mathbf{c}}_t + \widehat{\mathbf{c}}_{m+1} \end{aligned} \quad (22)$$

Suppose we have some surrogate loss function ℓ, we define the following loss function:

$$\begin{aligned} L &:= \sum_{k \in [N]} \sum_{j \in [C-1]} \ell(M(\boldsymbol{x}_k, j)) \\ &= \sum_{k \in [N]} \sum_{j \in [C-1]} \ell(-\epsilon \| \sum\nolimits_{t=1}^m \widehat{\mathbf{L}}_t + \widehat{\mathbf{L}}_{m+1} \|_q \\ &\quad + \sum\nolimits_{t=1}^m \widehat{\mathbf{c}}_t + \widehat{\mathbf{c}}_{m+1}) \end{aligned} \quad (23)$$

Note that all $\widehat{\mathbf{L}}_t, \widehat{\mathbf{c}}_t$ for $t = \{1, \cdots, m\}$ are precomputed and can be treated as constants, and $\widehat{\mathbf{L}}_{m+1}, \widehat{\mathbf{c}}_{m+1}$ are functions of the neural network parameters $\mathbf{W}_{m+1}, \mathbf{b}_{m+1}$ of model f_{m+1} (due to Prop. 3.1, see Wong and Kolter (2018); Zhang et al. (2018) for the explicit form). We can thus take the gradient $\frac{\partial L}{\partial \mathbf{W}_{m+1}}$, $\frac{\partial L}{\partial \mathbf{b}_{m+1}}$ and use typical gradient-based optimization tools to update model $f_{m+1}(x)$ and reduce the loss L. Compared to the setting in the previous section where each base model is trained independently and then fixed, in (23), model $f_{m+1}(x)$ knows the "weakness" of the ensemble of all previous models, and attempts to "fix" it, offering more flexibility.

4 Experiments

Overview and Setup. We evaluate the effectiveness of RobBoost, by using it to find the best ensemble in a relatively large pool of models on MNIST and CIFAR-10 datasets. Since we focus on improving certified robustness, our main metric to evaluate model robustness is *verified error* on test set, as defined in (6);